

제 3 장 파일 시스템(file system)

- 유닉스 파일 시스템은 파일과 디렉토리를 담고 있는 큰 개념

3.1 유닉스의 다중 사용자 환경 지원을 위한 파일 기능

(1) 사용자와 파일 소유권

- 유닉스의 모든 파일은 시스템 사용자 중 한 사람에 의해 소유권(ownership)이 결정

- 각 파일에 대한 소유권은 사용자 식별번호(user-id : UID)에 의하여 구분
- /etc/passwd 파일에 저장

```
sjjang:x:100:20::/home/sjjang:/bin/ksh
```

user name	password	사용자 식별번호	group id	comment	home directory	Shell 종류
-----------	----------	----------	----------	---------	----------------	----------

= login ID /etc/shadow 파일에 password를 저장 ---> 파일 보안 강화

- 파일의 소유권 변경

- 파일의 소유자

- super user -> 사용자 식별번호 변경 가능

- 파일의 그룹

- /etc/group에 정의
- gid에 의해 식별

UID(User Identification)

---> 사용자를 식별하는 고유한 값

file permission

---> (**~mask**)&mode

---> **mask : UNIX 시스템에서 제공하는 환경변수**

(2) 파일 허가과 파일 모드(file permission, file mode)

- 유닉스 시스템에서 파일의 소유권을 가진 소유자는 파일에 대한 허가(permission)를 선택
- ls 명령어의 -l 옵션을 사용

-rwxrwxrwx	2	sjjang	root	50	DEC 9	15:00	test
①	②	③	④	⑤	⑥	⑦	⑧

- ① 파일의 종류와 허가권 표시
- ② 링크수 - 원본파일을 포함해서 사본파일 개수
- ③ 파일의 소유권, 즉 파일을 만든 사용자
- ④ 사용자가 속한 그룹
- ⑤ 파일의 크기
- ⑥ 파일을 마지막으로 저장한 날짜
- ⑦ 파일을 마지막으로 저장한 시간
- ⑧ 파일 이름

* 허가상태(허가 스트링)

d	rwX	rwX	rwX
①	②	③	④

① 파일구분

- (대쉬) : 일반적인 파일

d : 디렉토리

l : 심볼릭 링크(링크파일)

b : 블록형 특수 파일 b : block special file

c : 문자형 특수 파일 c : character special file

2 : user - 파일의 소유자에 대한 허가권

- (대쉬) : 그 자리의 허가를 불허

- r(read) : 읽기 허가

- w(write) : 쓰기 허가

- x(execute) : 실행하기 허가

3 : group - 소유자가 속해있는 그룹에 대한 허가권

4 : other - 소유자와 그룹 이외의 사용자에게 대한 허가권

mask

---> unix 시스템의 환경변수

---> mask = 07

---> 파일의 생성 permission (mode)

또는 file permission 변경시

(\$ chmod mode aaa)

---> 실제 파일 permission은 ~mask & mode

---> mask가 07이라서 기타 사용자

(3) 수행 가능한 파일에 대한 그밖의 허가

● SetUID/SetGID 특수 권한(Special Permission) permission은 전부 off(---)로 생성된다.

● 프로그램을 실행할 때 실행한 사용자의 권한이 아니라 파일 소유자의 권한으로 실행되는 기능

● 중요하거나 민감한 자료에 대한 접근을 제한하고 보호할 수 있음

• 다른 사용자가 특정한 프로그램을 통해서만 파일에 접근

• passwd 프로그램

• - 일반 사용자가 실행하지만 root 권한으로 실행되어 /etc/shadow

같은 시스템 파일은 수정 가능 권한 표시

• - 허가권에서 "s" 문자로 표시 ("-rwsr-xr-x")

0644

(4) 파일 생성 마스크와 umask시스템 호출

~07&0644

♥ 파일 생성 마스크

Ex) fd = open("newfile", O_CREAT, 0655);

770 & 644

• fd = open(pathname, O_CREAT, mode);와 fd =

111 111 000

open(pathname), O_CREAT, (~mask)&mode); 는 동일한 의미

& 110 100 100

• * mask가 07 (04+02+01)인 경우

• fd = open ("newfile", O_CREAT, 0644);

110 100 000 ---> 0640

• 문장으로 생성된 파일의 허가권은 0640의 모드를 가진다.

0644(-rw-r--r--)

---> 0640(-rw-r-----)

● umask 시스템 호출

- 파일 생성 마스크의 변경

mask 환경변수는 000으로 반드시 셋팅
이 필요

- 사용법

umask() 시스템 호출 사용법
<pre>#include <sys/types.h> #include <sys/stat.h> mode_t umask(mode_t newmask);</pre>

● umask 시스템 호출을 이용한 프로그램 작성 예제

-> pp.86

- (중요) umask(07)의 경우 파일 permission이 어떻게 변하는지 ?

중간 시험 범위

(5) open() 시스템 호출과 파일 허가권

● open() 시스템 호출 사용법

open() 시스템 호출 사용법
<pre>#include <sys/types.h> #include <sys/stat.h> #include <fcntl.h> int open (const char *pathname, int flags, [mode_t mode]) ;</pre>

-pathname : 개방될 파일의 경로 이름

절대 경로명 (/usr/keith/junk) 나

상대 경로명 (junk)이 올 수 있다.

-flags : 접근 방식

#include <fcntl.h> 에 정의

O_RDONLY : 읽기전용

O_WRONLY : 쓰기전용

O_RDWR : 읽기, 쓰기 모드

optional

--> 사용해도 되고 안써도 된다.

--> -1 return

• 성공적인 open : 양의 정수의 값

• 오류 : -1

[mode_t mode]

--> file 생성 permission

-->

● 파일의 open에 대해서 복합적인 기능을 가지게 할 수 있다.

● `filedes = open(pathname, O_WRONLY | O_CREAT | O_TRUNC, 0600);`

~~프로세스가 쓰기 접근을 허용한다면 기존의 파일이 존재할 경우~~

~~파일을 없애버림~~

Close() 시스템 호출 사용법
<pre>#include <unistd.h> int close (int filedes) ;</pre>

O_CREAT

---> 파일이 생성된 적이 없

으면 새로운 파일을 생성

-->파일을 생성 : mode (파일 허가권)

`filedes = open("file", O_RDONLY);`

close() system call

---> open() system call

이 나타나면 close()

system call이 나타나야한다.

\$ man open

~~

main(){

---> open() 되어 있는 모든 file descriptor table entry를 free
(close() sytem 호출을 수행한 효과)

open(~~~);

} <-- process
zombie 상태 진입

close(filedes);

(5-1) read()시스템 호출 사용법

read()시스템 호출 사용법

#include <unistd.h>

ssize_t read (int filedes, void *buffer, size_t n);

- 첫번째 인수 : open이나 creat로부터 얻은 파일 기술자
- 두번째 인수 : 자료가 복사되어질 문자 배열의 포인터

[읽기-쓰기 포인터]

- 파일 foo 가 최소한 1024개의 문자열을 가지고 있다고 가정
- foo로부터 512문자열을 buf1에 저장, 다음 512문자열을 buf2에 저장

int fd;

ssize_t n1, n2;

char buf1[512], buf2[512];

. 파일 포인터를 이용하여 파일내의 데이터의 위치를 관리

. (중요포인터)파일 읽기/쓰기에서 파일 포인터 관리

. 임의로 파일 내의 위치를 변경하고자 할 경우 : lseek를 사용

if((fd = open("foo", O_RDONLY)) == -1)

return(-1);

n1=read(fd, buf1, 512); ---> foo 파일로부터 512바이트 크기의 데이터를 buf1으로 읽어들인다.

n2=read(fd, buf2, 512);

---> fd에서 offset =0

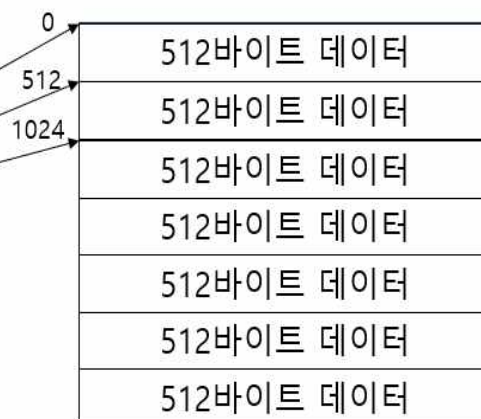
(초기값, 커널내에서 사용)

---> n1 : 512

---> offset = 0 + n1(512) =512

---> offset = 512 + n2(512) = 1024

File pointer



READ() 시스템 호출의
RETURN VALUE

---> -1 : FAIL

---> 양의 정수값 : 커널이 파일로
부터 읽은 바이트 수

세번째 아규먼트

---> 커널로부터 (파일에서) 읽기
요구를 하는 총 바이트 수

첫번째 인수 : OPEN이나 CREAT로

(5-2) write()시스템 호출 사용법

write()시스템 호출 사용법

```
#include <unistd.h>
```

```
ssize_t write (int filedes, const void *buffer, size_t n) ;
```

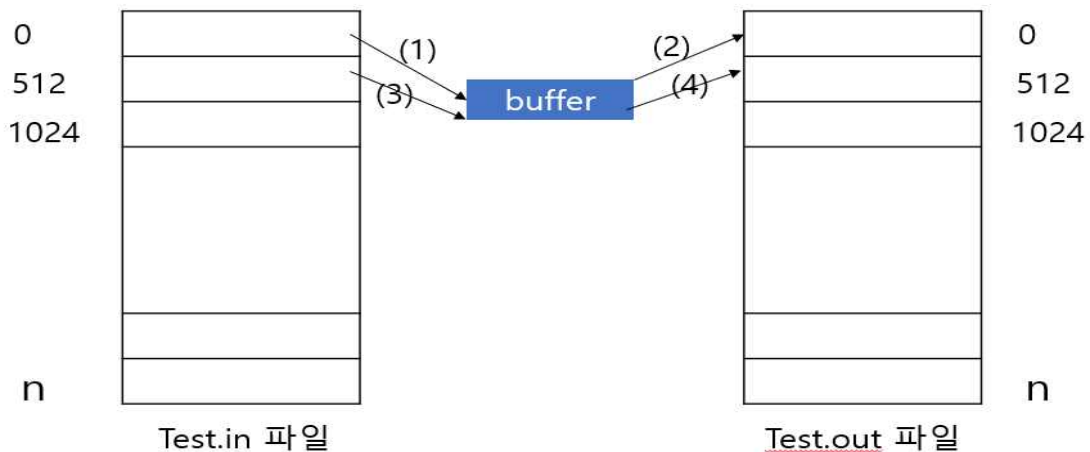
- filedes : 파일 기술자
 - buffer : 쓰여질 데이터에 대한 포인터
 - n : 쓰여질 바이트의 양
- 복귀값 : 쓰여진 문자의 수 or -1

(5-3) open()/read()/write()시스템 호출을 이용한 예제

- (중요) 아래 시스템 호출의 경우 동작 과정 설명

```
//infile : test.in, outfile:test.out
while(( nread = read(infile, buffer, BUFSIZE)) >0 )
{
    if (write (outfile, buffer, nread) < nread )
    {
        //프로그램 종료 코드
    }
}
```

- 위 read/write 시스템 호출에 대한 동작 과정 그림



- 이 프로그램 동작과 동일한 유닉스 명령어 :

3.2 하나의 파일에 대해서 다수의 이름을 갖는 파일

- 유닉스 파일은 하나 이상의 이름으로 식별될 수 있다.

(1) link() 시스템 호출

- 사용법

link() 시스템 호출 사용법
<pre>#include <unistd.h> int link(const char *original_file, const char *new_link_file);</pre>

(2) unlink() 시스템 호출

- ex) unlink("/home/sjjang/1");
- 링크계수(link count)를 하나 감소

(3) rename() 시스템 호출

- 정규 파일, 디렉토리 이름 바꾸는데 사용

rename() 시스템 호출 사용법
<pre>#include <stdio.h> int rename(const char *old_file_name, const char *new_link_file);</pre>

3.3 파일 관련 정보의 획득을 위한 stat(), fstat() 시스템 호출

Stat(), fstat()시스템 호출 사용법
<pre>#include <sys/types.h> #include <sys/stat.h> int stat(const char *file, struct stat *buf); int fstat(int fd, struct stat *buf);</pre>

- <sys/stat.h> 에서 stat 구조체의 구조
- pp. 97 - 98

- stat 구조체 정리

dev_t	st_dev;
ino_t	st_ino;
mode_t	st_mode;
nlink_t	st_nlink;
uid_t	st_uid;
gid_t	st_gid;
dev_t	st_rdev;
off_t	st_size;
time_t	st_atime;
time_t	st_mtime;
time_t	st_ctime;
long	st_blksize;
long	st_blocks;

- (중요) stat 구조체 자료 구조 설명

요소	기능
st_dev, st_ino	이것은 st_dev와 함께 한 파일을 유일하게 식별한다. 이 두 자료 구조는 유닉스 파일구조를 관리하는데 근본적으로 관련된다.
st_mode	프로그래머가 파일에 연관된 허가를 계산할 수 있도록 하여 준다.
st_nlink	이 값은 link와 unlink시시스템 호출시마다 값이 변경된다.
st_uid, st_gid	초기값은 creat() 시스템 호출이나 open() 시스템 호출로써 지정된다.
st_rdev	이것은 파일 엔트리가 장치를 기술하는데 사용될 때만 의미를 가진다.
st_size	파일이 디스크 상에 저장될 때 디스크의 물리적 영역에 위치하게 되고 이 파일의 크기 정보를 담고 있다.
st_atime	이 자료 구조는
st_mtime	이 자료 구조는 특히, 파일에 쓰기 명령이 있을 때마다 이 자료구조의 값이 변경된다.
st_ctime	이 자료 구조는 이것을 변경하는 시스템 호출은 link(), chmod(), write()등이 있을 수 있다.
st_blksize	이 자료 구조는 이 파일을 위한 파일 시스템 고유의

st_blocks	이 자료 구조는 이 파일에 할당된 물리적
-----------	------------------------

● stat 시스템 호출을 사용한 프로그램 예제

-> pp.99~100

-> 파일 이름의 list가 주어져 있을 경우 매 분마다 리스트에 있는 어떤 파일이 변경 되었는지를 조사 : pp.101~102

● Background process

사용자가 수행하는 작업을 동시에 실행할 수 있도록 해준다.

- ➔ Parallel processing이 가능
- ➔ 작업 처리 속도가 빠르다.

● Foreground process 개념

Foreground process

- ➔ 사용자가 수행하는 작업을 마쳐야 그 다음 작업을 수행할 수 있다.
- ➔ 여러 개의 작업을 동시에 실행하는 것이 안된다.

<참고 자료>

<<umask 시스템 호출 예제>>

```
#include <fcntl.h>
#include <sys/stat.h>

main()
{
    specialcreat("aaa", 0644);
}

int specialcreat(const char *pathname, mode_t mode)
{
    mode_t oldu;
    int filedес;

    if((oldu = umask(07)) == -1)
    {
        perror("saving old mask");
        return(-1);
    }

    if ((filedes = open (pathname, O_WRONLY | O_CREAT | O_EXCL,
mode)) == -1)
        perror("opening file");

    if (umask (oldu) == -1 )
        perror("restoring old mask");
    return filedес;
}
```

<<read/write system 호출 예제>>

```
#include <unistd.h>
#include <fcntl.h>

#define BUFSIZE 512
#define PERM    0644

int copyfile (const char *name1, const char *name2)
```

```

{
    int infile, outfile ;
    ssize_t nread;
    char buffer[BUFSIZE];

    if ((infile = open (name1, O_RDONLY )) == -1)
        return (-1);

    if ((outfile = open(name2,O_WRONLY | O_CREAT |O_TRUNC,PERM)) ==
-1)
    {
        close(infile);
        return (-2);
    }

    while(( nread = read(infile, buffer, BUFSIZE)) >0 )
    {
        if (write (outfile, buffer, nread) < nread )
        {
            close(infile);
            close(outfile);
            return (-3);
        }
    }

    close(infile);
    close(outfile);

    if ( nread == -1)
        return (-4);
    else
        return (0);
}

```

```

#include "2-5.h"

main()
{
    copyfile("test.in","test.out");
}

```